FINAL DELIVERABLE

Northstar

Task Manager Assessment

GiaHuy Pham | November 9, 2026

Solution overview

Northstar is a shared-group Kanban task manager. Users can begin as anonymous guests, register that same identity with email and password, and collaborate through accepted group invitations. Each account starts with a private group and can create or join more groups. The board supports task creation, editing, deletion, and movement through To Do, In Progress, In Review, and Done.

The solution uses React and TypeScript with Vite, Supabase Auth/PostgreSQL/Realtime, and a black-and-green interface. It combines real-account assignments, labels, comments, activity history, due-date indicators, filtering, and group-wide summaries. Authorization is enforced in the database; live notifications refresh authoritative data while preserving active drafts.

Live frontend

`https://northstar-six-zeta.vercel.app/`

GitHub repository

`https://github.com/wolflieu201105/northstar`

Document guide

Architecture and design decisions 2

Database model and integrity 3

Advanced features and behavior 4

Local setup and deployment 5

Verification, tradeoffs, and improvements 6

Database relationships 7

Architecture & design decisions

A browser frontend uses Supabase directly through the native SDK. PostgreSQL owns the shared data, constraints, permissions, and transactional mutation boundary. Vercel serves the compiled frontend.

React interface

Feature modules and scoped state

Auth + checked API

Native identities and atomic RPCs

Postgres + Realtime

Membership rules and live signals

Group ownership rather than a manager-owned board

Accepted members share editing and invitation powers. The owner controls member removal, ownership transfer, and group deletion. Assignments reference real accepted accounts; typing a name never creates an artificial user.

Native identity with low-friction guest entry

Supabase supplies anonymous and registered Auth users. The application stores public profile data separately. Registration links credentials to the current guest UUID; deliberate login to another account switches identity rather than merging unrelated work.

Database-enforced authorization and integrity

RLS restricts reads to permitted memberships. Self-profile updates expose only editable fields; other writes use invoker RPC wrappers and narrowly checked private functions. Composite foreign keys reject task, label, and member relationships from another group.

Atomic saves and explicit conflicts

Task fields, assignments, labels, revision tokens, and generated activity commit together. Exact PostgreSQL timestamps detect stale changes. Public revision errors return HTTP 409/PT409, preserving a draft for review instead of silently overwriting newer work.

Draft-safe live collaboration

Private Broadcast channels carry content-free invalidation hints, not task/comment snapshots. The client refetches under current RLS, coalesces bursts, and reconciles after reconnect, focus, or a visible 30-second interval. Losing membership clears inaccessible views.

Feature-oriented code and predictable navigation

Small components, services, hooks, stores, shared types, and plain-CSS files are organized by feature. Query-string group/task/filter links support direct navigation and Back/Forward. Identity/group isolation and navigation guards protect drafts and prevent cache mixing.

Database model & integrity

The application defines ten public tables and one private quota table. Supabase manages auth.users and realtime.messages; the application references those services instead of reimplementing passwords or transport storage. See the relationship diagram on page 7; complete SQL remains in supabase/schema.sql.

Table	Purpose	Keys / relationships
`public.profiles`	Public identity: username, name, color.	PK id → auth.users; unique normalized username.
`public.groups`	Shared container and current owner.	PK id; owner → profile and a deferred active-owner membership FK.
`public.group_members`	Accepted membership and active/left state.	PK (group_id, user_id); links group to profile; active-state uniqueness.
`public.group_invitations`	Recipient requests with pending/terminal state.	PK id; group, sender membership, recipient profile; one pending request per pair.
`public.tasks`	Title, description, status, priority, due date, revision.	PK id; group FK; unique (group_id,id); creator snapshot retained.
`public.labels`	Named colored group labels and archive state.	PK id; group FK; unique group/name, including archived names.
`public.task_assignments`	Many-to-many task / real-account links.	PK (group_id,task_id,assignee_user_id); same-group task and active membership FKs.
`public.task_labels`	Many-to-many task / label links.	PK (group_id,task_id,label_id); composite same-group task and label FKs.
`public.comments`	Append-only task discussion with author snapshot.	PK id; same-group task FK; body 1–2,000 characters and nonblank.
`public.task_activity`	Database-generated event payload and actor snapshot.	PK id; same-group task FK; validated event types; chronological index.
`private.api_rate_limits`	Bounded invitation and account-lookup quotas.	PK (user_id,bucket,window_start); no direct API grants.

Integrity and retention

Tasks belong to one group. Assignment membership must be active, and a group must retain an active owner. Pending invitations provide a minimal preview but no board access. Invitations expire after seven days; terminal retries cannot recreate a removed membership.

Task/group deletion cascades to dependent links, comments, and activity. Member departure clears assignments and preserves shared work. Non-owner account deletion retains saved author/actor names; an owner must transfer or delete owned groups first.

Advanced features built

The core board includes persistent task CRUD, priority, due dates, drag-and-drop status changes, rollback on failed moves, and a status selector for keyboard/mobile use. The following enhancements extend that workflow.

Enhancement	How it works
Team & assignments	Members are actual accepted users or guests. Tasks support multiple assignees; cards show initials/color avatars. Removing a member clears current assignments without deleting shared tasks or historic names.
Task comments	The task detail panel provides a separate comments table, chronological paging, timestamps, real author names, blank/length validation, retained drafts, and stable request IDs for duplicate-response recovery.
Task activity log	Triggers record creation, status changes, field edits, assignments, and label changes. Activity shows old/new values and saved actor/relation names. No-ops and failed saves produce no success history.
Labels / tags	Create, rename, recolor, archive, or restore group labels. Attach multiple labels and filter by them. Existing archived links remain visible; newly attaching an archived label is rejected.
Due-date indicators	Cards distinguish overdue, due today, and upcoming work using local calendar dates. Indicators refresh at midnight or page resume. Completed tasks do not appear overdue.
Search & filtering	Search matches title/description without case sensitivity. Combine priority, assignees, and labels: ANY within a multi-select, AND across filter categories. Unassigned/Unlabeled modes are available.
Board summary	Total, completed, and overdue counts cover the whole selected group. A separate filtered result count makes the active view clear without changing the underlying board.
Accounts & groups	Email/password login, guest registration, recovery, unique usernames, account IDs, group switching, and invitation acceptance support collaboration. Owner-only actions remain separate from shared task editing.
Live synchronization	Private account/group channels trigger authoritative background reads. Coalescing, reconnect/focus/poll recovery, exact revision conflicts, and scoped caches preserve drafts and prevent cross-group data mixing.

Example flow: create a group → invite an existing account → recipient accepts → assign both members → comment and move a task → inspect Activity → refresh or view the update from the other account.

Run locally & deploy

1. Prerequisites

Use Node.js 24 (the repository .nvmrc and CI major), npm 11, Git, and a Supabase project. The application uses pinned package versions and package-lock.json.

2. Prepare an empty Supabase project

Enable anonymous sign-ins, Email/Password, and Allow manual linking. In SQL Editor, run the complete supabase/schema.sql once on an empty application schema. It includes the group model, secure APIs, private Realtime, and conflict fix. The complete file is the single database setup source; no historical migration or verification scripts are required.

Use private Realtime channels (disable Allow public access). Add http://localhost:5173/** to allowed Auth redirects. On a new local-only project, set the Site URL to localhost; when sharing the existing deployed project, retain its production Site URL and add the local redirects. Native confirmation/recovery follows the project settings.

3. Clone and configure

```
git clone https://github.com/wolflieu201105/northstar.git
cd northstar
npm ci
Copy-Item .env.example .env.local
```

The copy command above is PowerShell; use cp .env.example .env.local in bash. Edit .env.local with your own public project values:

```
VITE_SUPABASE_URL=https://your-project-ref.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_your_key_here
```

Only a browser-safe publishable key is needed. Keep privileged credentials and .env.local out of Git. VITE_* values are visible in the browser bundle.

4. Start and verify

```
npm run dev
# Open http://localhost:5173
npm run lint
npm test -- --maxWorkers=2
npm run build
npm run preview
```

5. Deploy the frontend

Import the repository into Vercel using the Vite preset, npm ci, npm run build, and dist output. Set the same two public environment variables before building. Configure the stable production Site URL and account upgrade/recovery redirects in Supabase; the README lists the exact live callbacks.

Verification, tradeoffs & improvements

Recorded evidence	Result
Frontend baseline	187 tests in 35 files, lint, TypeScript, and production build passed.
Database / HTTP	Local fresh/reset parity and three concurrent races passed. The native hosted suite passed 189 guest/API/Realtime assertions, including all three HTTP 409 conflicts and unchanged-data/history readbacks.
Production use	The live page/assets returned HTTP 200. The project owner confirmed production sign-in, persistence after refresh, and password recovery working.

Engineering tradeoffs

Decision	Benefit and constraint
Direct Supabase client	A small frontend and managed backend reduce custom server code. Correct RLS, grants, relationship constraints, and checked RPCs are therefore essential.
Serialized group writes	A parent-group lock keeps membership changes and saves consistent for small groups. Larger organizations would benefit from finer-grained locking and measured load testing.
Client-side board filters	Filtering and summaries are straightforward for a loaded group board. Large datasets would need server-side search, paging, and aggregate counts.
Invalidation-based live data	Content-free hints keep records behind RLS and allow safe recovery. Refetching and periodic reconciliation trade some network work for consistency and simpler conflict handling.

What I would improve with more time

Add browser end-to-end coverage for registered-user isolation, exact guest-to-account preservation across devices, concurrent dirty drafts, access loss, and mobile/keyboard flows. Repeat installation against a fresh hosted test project and capture accessibility/performance baselines.

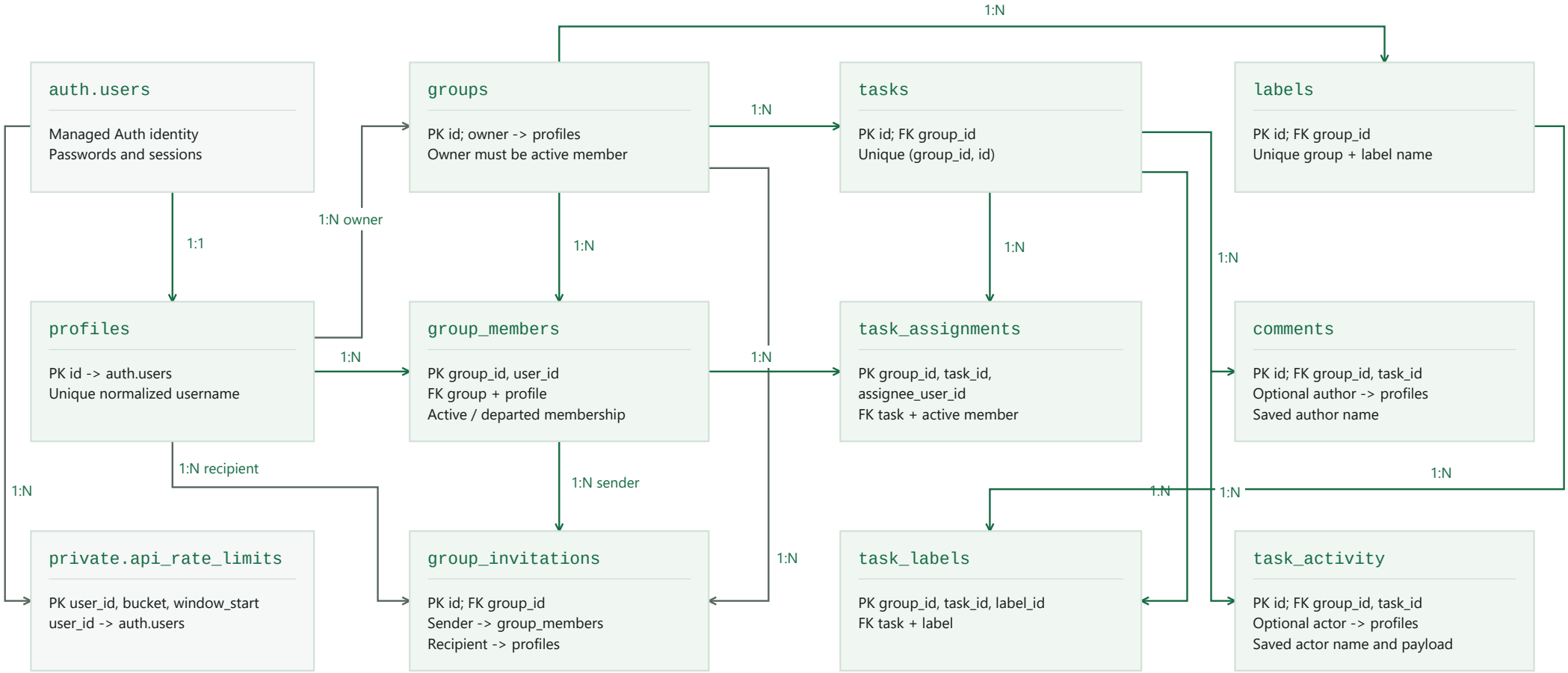
Add persistent ordering within columns, file attachments/avatar uploads, invitation emails, optional OAuth, and deliberate offline write synchronization. Profile-based initials/color avatars and in-app invitation requests currently cover the implemented scope.

Guests cannot recover an identity after losing its browser session unless they registered first; signing into an existing account does not merge guest data. The recorded review also leaves leaked-password protection disabled and a development-only minimatch advisory for follow-up. The last production-dependency audit had no findings.

Evidence boundary: native API checks used guest tokens. Registered-user API repetition and the remaining detailed browser/mobile cases were not completed; they are future verification work, not reported passes.

Database relationships

Arrows run from a referenced parent to its referencing child. 1:N = one-to-many; 1:1 = one-to-one. Unqualified table names are in public.



Integrity: composite group/task/label keys reject cross-group links. Assignments require an active member; every group owner must remain an active member. Pending invitations do not grant membership.

Additional profile references: task/label creator, membership inviter, and assignment assigner. Nullable author/actor references preserve saved names after account deletion. Task/group deletion cascades to dependent content.

`Full SQL: https://github.com/wolflieu201105/northstar/blob/main/supabase/schema.sql`